Deep Learning · Entregable final

Reporte de mejoras y propuesta de profundización con LLMs y agentes

Aplicado al TFM Tema 50: *Integración y consolidación territorial de datos abiertos gubernamentales del Ecuador para la toma de decisiones*

Wilson Paredes
Diana Tandalla
Alexander Luna
Bryan Sarmiento

Maestría en Ciencia de Datos y Máquinas de Aprendizaje · UIDE
Quito, octubre de 2026

Notebooks de soporte: `Entregable_Final_Mejoras_TFM_Tema50.ipynb` y `W2_Mejora_Pruebas_Reglas_Distintas.ipynb` (recalculan el antes y el después de cada cambio).

Parte 1. Reporte de correcciones y mejoras

Las tres entregas trabajan con datos propios del TFM. La 1 y la 3 entrenan un modelo supervisado que reproduce el consenso de tres detectores de anomalías (Isolation Forest, LOF y One-Class SVM) sobre la matriz cantón-año; la 2 normaliza nombres de cantón mal escritos con redes recurrentes y de atención a nivel de caracteres. Cada observación del profesor se documenta con su número original. Todas las cifras de antes y después se recalculan en los notebooks de soporte o se leen de los archivos de resultados de cada entrega; ninguna se transcribe a mano.

1.1 Entrega 1: perceptrón multicapa (MLP)

Tabla 1. Observaciones recibidas en la entrega 1 (MLP).

Estado anterior (qué había antes)	Cambio realizado (qué se cambió)	Justificación (por qué se cambió)	Efecto / impacto visto (métrica o resultado)
Observación 1: los valores del notebook y del informe no cuadran exactamente (votos y algunas métricas)			
Las cifras se copiaban a mano del notebook al informe.	Cada notebook guarda sus resultados en un JSON (`resultados_*.json`) y el informe se genera desde ese archivo; se aplica desde la entrega 2.	Elimina errores de transcripción y permite volver a generar el informe al cambiar el modelo.	Cero cifras transcritas a mano. Al recalcular en este entregable, los RMSE coinciden exactamente y el F1 en cantones nuevos difiere en 0,006.
Observación 2: antes de seguir mejorando, revisar qué son las anomalías y obtener retroalimentación experta			
Se afinaba el modelo sin saber qué significaban los casos marcados.	Dossier de los 56 casos (consenso y top-30) con la serie de cada cantón, triage automático por reglas y hoja de revisión para expertos con listas desplegables.	Una métrica alta no dice si los casos sirven; fuera del ejercicio académico importan el uso y la veracidad del resultado.	De 56 casos, 12 se explicaban por tamaño del cantón, 14 por un indicador casi constante (IESS) y solo 10 no tenían explicación simple; 15 del top-30 eran Quito, Guayaquil o Cuenca. Esto motivó las mejoras de la sección 1.4. La revisión por expertos externos sigue pendiente.
Observación 3a: ¿qué sentido tiene el KNN en la imputación y por qué k = 5?			
Faltantes imputados con `KNNImputer(n_neighbors=5)`, sin justificar el método ni el valor de k.	Se compararon imputadores ocultando el 10 % de los valores observados (10 repeticiones) y se adoptó la mediana de los demás años del mismo cantón: `KNNImputer(5)` → `ImputadorHistoria().fit(X_train)`.	Con 4 indicadores poco correlacionados, «filas parecidas» aporta poca información; el nivel propio del cantón explica la mayor parte de su valor en cualquier año. Un método debe superar al menos a la media de la columna.	RMSE de recuperación: **1,414 → 0,737** (−48 %). KNN con k = 5 era incluso peor que la media de la columna (1,335).
Observación 3b: se usan todos los datos y luego se divide (información de prueba en el entrenamiento)			
El imputador, el escalado y los tres detectores que generan la etiqueta se ajustaban con todas las filas; después se dividía en entrenamiento y prueba.	Todo se ajusta solo con entrenamiento; la etiqueta de prueba se obtiene aplicando los detectores ya ajustados a la prueba: `LocalOutlierFactor(novelty=True)`, `IsolationForest.predict(X_test)`.	Si la etiqueta de prueba depende de las propias filas de prueba, el modelo se evalúa con información que no tendría en producción y la métrica es optimista.	Mismo MLP y mismas 20 divisiones: F1 **0,70 → 0,65** y PR-AUC **0,85 → 0,78**. La versión anterior sobrestimaba el desempeño en ≈0,05 de F1; la cifra nueva es la honesta.
Observación 4: evaluar la generalización a cantones nuevos con agrupación territorial			
Solo división de filas al azar: un mismo cantón aparecía en entrenamiento y en prueba con años distintos.	Se agregaron divisiones por grupos: `GroupShuffleSplit(groups=canton)` (20 % de cantones con todos sus años) y por provincia; 20 repeticiones cada una.	El uso real del modelo es puntuar territorios que no vio; con filas al azar puede «memorizar» el cantón.	F1: filas al azar **0,65**, cantones nuevos **0,63**, provincias nuevas **0,54**. La caída es pequeña con cantones nuevos y mayor con provincias; la variabilidad es alta (±0,2).

Nota. Cifras recalculadas en las secciones A1 a A4 del notebook Entregable_Final_Mejoras_TFM_Tema50.ipynb, sobre la matriz de esa entrega (1.284 cantón-año × 4 indicadores).

1.2 Entrega 2: BiGRU y Transformer para normalizar nombres de cantón

Tabla 2. Observaciones recibidas en la entrega 2 (BiGRU y Transformer).

Estado anterior	Cambio realizado	Justificación	Efecto / impacto visto

(qué había antes)	(qué se cambió)	(por qué se cambió)	(métrica o resultado)
Observación 1: la prueba sintética es optimista porque entrenamiento, validación y prueba siguen las mismas reglas de cambio			
Las tres particiones salían del mismo generador con las 5 familias de error. Exactitud sintética: BiGRU 0,969, distancia de edición 0,940.	Prueba con reglas distintas (*leave-one-family-out*): se entrena y valida con 4 familias y se prueba con la quinta, generada con otra semilla y sin textos compartidos; 5 rondas con el mismo código y la misma BiGRU de la entrega.	En el data lake aparecen errores que el generador no previó; la prueba debe medir errores que la red nunca vio.	Exactitud media de la BiGRU: **0,975 con la misma regla → 0,771 con la regla nueva**; la peor familia es «tokens» (0,255). La distancia de edición, que no aprende reglas, obtiene 0,895 en las mismas pruebas.
Observación 2: el conjunto real es muy pequeño y la ganancia de estas arquitecturas frente a la distancia es poca			
Prueba real de 38 cadenas: BiGRU y distancia aciertan 38/38; Transformer 36/38. La ventaja de la red se apoyaba en la prueba sintética (F1 0,965 frente a 0,942).	(1) Intervalo de Wilson y prueba de McNemar para cuantificar la incertidumbre. (2) Cascada: distancia de edición si la similitud es ≥ 0,80; si no, BiGRU. (3) Ampliación de la prueba real: hoja de etiquetado experto con los 85 textos que nada resuelve (102.989 registros) y script que calcula la exactitud al llenarla.	Con 38 cadenas un acierto vale 2,6 puntos y todos los métodos empatan; la decisión sobre qué usar en la tesis debe apoyarse en evidencia suficiente.	IC 95 % para 38/38: **[0,908; 1]**: la prueba no distingue métodos; harían falta ≈729 cadenas por método para separar 0,97 de 0,94. En la regla nueva, la red acierta sola 154 textos y la distancia sola 4.210 (McNemar significativo en 4 de 5 familias). La cascada (0,809) no mejora a la distancia sola y se descarta. En los 85 textos reales difíciles, BiGRU y distancia coinciden solo en 26: la prueba de 38 cadenas no refleja la dificultad real.

Nota. Cifras de resultados_W2.json (entrega 2) y de resultados_W2_mejoras.json, generado por W2_Mejora_Pruebas_Reglas_Distintas.ipynb.

Conclusión de la entrega 2. La ventaja de la BiGRU sobre la distancia de edición desaparece cuando el error no se parece al del entrenamiento: con reglas nuevas cae 0,20 puntos de exactitud, mientras la distancia se mantiene en 0,895. Ambos métodos fallan con los prefijos institucionales (familia «tokens»: 0,255 y 0,485) y con los textos que no son cantones. Para la tesis se adopta la distancia de edición como método principal, con una regla que quite prefijos como «CANTÓN» o «CABECERA CANTONAL», una clase «no es cantón» y revisión humana de los casos de baja similitud.

1.3 Entrega 3: transfer learning, fine-tuning e interpretabilidad

Estado anterior (qué había antes)	Cambio realizado (qué se cambió)	Justificación (por qué se cambió)	Efecto / impacto visto (métrica o resultado)
Observación 1: Correcto uso de TabPFN y explicabilidad; analizar a fondo la influencia de la información imputada	Se implementaron máscaras binarias de imputación (is_imputed) como características de entrada adicionales y se realizó un análisis comparativo SHAP diferenciado (*Valores Observados vs. Valores Imputados*). Adicionalmente, se aplicó un filtro de evidencia observada que descarta alertas donde la mayor desviación provenga de un valor imputado.	La explicabilidad demostró que el modelo estaba asignando importancia predictiva a artefactos generados por el algoritmo de imputación (copia de valores extremos entre años). Un modelo de apoyo a la decisión no puede alertar anomalías basadas en datos reconstruidos.	La proporción de alertas del Top-30 explicadas por valores imputados se redujo de **15 a 0 casos** (0%). La estabilidad de las detecciones (índice Jaccard en submuestras) alcanzó **0.75** (superando el umbral de 0.70). Se comprobó que el *fine-tuning* parcial de TabPFN pasa de un F1 de 0.25 (MLP base) a 0.75 en cantones nuevos y 0.66 en provincias nuevas.

Se observó en el análisis explicativo global que los datos imputados mediante KNN/Mediana ejercían una influencia desproporcionada en las predicciones y alertas del modelo.

1.4 Mejoras adicionales surgidas de la revisión del modelo

La revisión de casos de la observación 2 de la entrega 1 y la del TFM mostraron que el modelo detectaba sobre todo «cantones grandes» y artefactos de publicación. Se aplicaron estos cambios:

Tabla 3. Mejoras derivadas de la revisión de casos y del TFM.

Estado anterior (qué había antes)	Cambio realizado (qué se cambió)	Justificación (por qué se cambió)	Efecto / impacto visto (métrica o resultado)
Normalización por población			
Indicadores en log(1 +	Tasa por mil habitantes (Censo	El modelo debe detectar	Casos de Quito, Guayaquil o

conteo): el número de registros publicados crece con el tamaño del cantón.	2022) dividida por la tasa nacional del año: cociente de localización.	perfiles atípicos en relación con el tamaño, no «es una ciudad grande».	Cuenca en el top-30: **15 → 1**.
Matrices de presencia y de valores			
Matriz única con ceros: un cero mezclaba «no se publicó» con «no hubo eventos».	Matriz de presencia (qué se publica) separada de la de valores (qué dicen los datos), con los ceros como faltantes.	El puntaje ordenaba los cantones por cantidad de información publicada.	Correlación del puntaje con la cobertura de la fila: **0,82 → 0,10** (umbral $\|\rho\| \leq 0{,}30$).
Indicador constante excluido del modelo de contenido			
La encuesta del Seguro Social Campesino (IESS) era una de las 4 variables.	Se excluyó del modelo y se dejó como control de calidad de la publicación.	Toma el mismo valor en casi todos los cantones: mide la estructura del archivo, no el territorio.	Mediana de 97,1 % de cantones con el mismo valor cada año; dominaba 14 de los 56 casos revisados.
Ampliación de la matriz			
Umbral de cobertura calculado sobre 8 años: quedaban 4 indicadores de 4 instituciones.	Umbral aplicado sobre una ventana de años (2021–2025), elegida con un criterio fijado antes de modelar.	La mayoría de los conjuntos cubre más del 75 % de los cantones, pero solo se publica 1 a 4 años.	Indicadores / instituciones: **4 / 4 → 10 / 8**; matriz de 807 × 10 con 62 % de densidad.
Filtro de evidencia observada			
Casos priorizados por puntaje aunque la desviación estuviera en un valor imputado.	Solo entran a la lista los casos cuya mayor desviación es un valor observado.	La imputación copiaba el nivel extremo de un año a los años sin dato.	Casos del top-30 explicados por un valor imputado: **15 → 0**. Cantones distintos en la lista: 19 → 25. Estabilidad (Jaccard en submuestras) 0,75, sobre el umbral de 0,70.
Modelo pre-entrenado frente al MLP desde cero (entrega 3)			
MLP de 2 capas (16 y 8) entrenado desde cero.	TabPFN v1 pre-entrenado; fine-tuning de las 2 últimas capas y del decodificador (4,7 M de 25,8 M parámetros).	Con pocas anomalías etiquetadas, un modelo pre-entrenado con millones de tablas generaliza mejor que una red entrenada desde cero.	F1 en cantones nuevos: **0,25 → 0,75**; en provincias nuevas: 0,27 → 0,66. SHAP mostró que 1 de cada 4 alertas dependía de valores imputados.

Nota. Cifras de los archivos resultados_cap3_v2.json, resultados_cap4.json y resultados_trabajo3.json, leídos en las secciones B y C del notebook de soporte. Las filas de esta tabla usan la matriz ampliada (807 × 10), salvo la de normalización, que compara los mismos indicadores antes y después.

Balance. Las correcciones bajaron algunas métricas, como el F1 de la observación 3b de la entrega 1 o la exactitud de la BiGRU con reglas nuevas, porque las cifras anteriores eran optimistas, y subieron otras porque el modelo dejó de aprender artefactos del dato. Las mejoras más grandes vinieron de cambiar los datos que entran al modelo y de partir de un modelo pre-entrenado, no de ajustar la arquitectura de la red original. Queda pendiente la validación de los casos con expertos, que es la única evidencia de utilidad real.

Parte 2. Innovación en la vida real: integración de LLMs y agentes

1. Uso de los conceptos del curso en la vida laboral o profesional

¿Qué problema específico pueden abordar y cómo?

Como Programador Analista en el Registro de la Propiedad y Mercantil de Rumiñahui, mi trabajo consiste en gestionar bases de datos SQL Server, servidores Java EE (WildFly) y garantizar el correcto funcionamiento de las plataformas registrales. Un desafío clave es la recepción de trámites con información contradictoria, nombres de titulares variados, linderos inconsistentes, descripciones poco claras o actos jurídicos contradictorios, así como la detección de inscripciones anómalas en grandes volúmenes de datos. A lo largo del curso se analizaron conceptos aplicables en dos áreas. Redes Neuronales Recurrentes (BiGRU) y Distancia de Edición, estas facilitan la generación de microservicios que estandarizan y normalizan información como parroquias, sectores y linderos en tiempo real, minimizando errores e inconsistencias al registrar solicitudes.

Modelos Tabulares Avanzados (TabPFN) y Explicabilidad (SHAP): se utilizan como instrumentos de auditoría en bases de datos para identificar patrones anómalos, como incrementos inusuales en prohibiciones, gravámenes o cifras atípicas, y aclarar los resultados con valores SHAP antes de emitir certificados o justificaciones de inscripción.

2. Incorporación de LLMs y agentes

¿Cómo puede un agente resolverlo mejor?

En la gestión universitaria tradicional suele pasar que los sistemas funcionan como islas de información aisladas, y un sistema agéntico viene a resolver esa fragmentación actuando como una capa inteligente superior: consulta al mismo tiempo plataformas académicas, financieras y de bienestar sin necesidad de reemplazar la infraestructura existente, conecta datos estructurados de bases relacionales como calificaciones y asistencias con documentos no estructurados PDF de syllabus, certificados médicos o reglamentos y, además, monitorea de forma continua la trayectoria académica para intervenir automáticamente antes de que ocurra la deserción estudiantil.

¿Qué funcionalidades adicionales tendría?

La incorporación de agentes mejora la operación institucional en cuatro frentes: en alerta temprana y bienestar, cruza el rendimiento en tiempo real con asistencias y test vocacionales y, cuando detecta riesgo alto, agenda tutorías por su cuenta y arma planes de acompañamiento; en matriculación y cobros, valida requisitos académicos, pagos y cruces de horarios, y resuelve inconsistencias sugiriendo paralelos o precalificando solicitudes de beca; en distributivos y horarios, analiza la disponibilidad docente, los perfiles de talento humano y la demanda de matriculación para proponer distributivos sin cruces de aulas; y en reorientación vocacional asistida, compara los resultados vocacionales iniciales con el historial real para recomendar adaptaciones curriculares o cambios de carrera a tiempo.

¿Cómo se implementaría?

Para garantizar seguridad, escalabilidad y privacidad de la información universitaria, la arquitectura se apoya en cuatro pilares: un despliegue on-premise o híbrido privado, con modelos de lenguaje de código abierto como Llama 3 o Qwen corriendo en contenedores Docker dentro de la infraestructura de la universidad; conectividad RAG y API REST, interactuando con bases de datos relacionales como PostgreSQL mediante servicios web seguros y usando bases de datos vectoriales para consultar la normativa vigente; una estructura multiagente especializada, coordinada por un agente supervisor que delega tareas a agentes de las áreas Financiera, Curricular y de Bienestar; y una gobernanza con supervisión humana (Human-in-the-Loop), donde la validación de identidad se hace con tokens JWT (OAuth2 / SAML / Active Directory) y la aprobación final de los trámites críticos siempre requiere la confirmación del funcionario o director de carrera.

3. Reflexiones finales

Desafíos que tuvieron y cómo los superaron; aspectos o temas que más les sorprendieron.

Desafíos que superamos: El mayor dolor de cabeza técnico del proyecto fue manejar con cuidado la fuga de información, también conocida como *data leakage*. Nos costó entender que el preprocesamiento, por ejemplo la imputación k-NN, podía cambiar de forma importante la distribución de los datos y generar artefactos que confundían a las redes neuronales. Lo resolvimos separando estrictamente los transformadores en los conjuntos de entrenamiento y de evaluación. También revisamos que el modelo generalizara bien por grupos territoriales.

Lo que más me sorprendió: Lo más interesante del curso fue darme cuenta del poder de las técnicas de interpretabilidad, como SHAP. También me sorprendió mucho la versatilidad de modelos preentrenados como TabPFN. Descubrir que uno de cada cuatro avisos dependía originalmente de un valor imputado me mostró que la explicabilidad no es solo una idea teórica, sino una herramienta muy útil para el control de calidad. De hecho, es clave antes de usar cualquier modelo de inteligencia artificial en un entorno real o gubernamental.

Anexo. Informes individuales

https://github.com/Alexander-Luna/NeuralNetworks-DeepLearning